OfflineID, Read Me First

NHAI Hackathon 7.0 entry for *"Develop a mobile based secure offline facial recognition and liveness detection system for remote locations."*

OfflineID is a React Native module that authenticates field personnel with **face recognition + liveness detection, fully offline** (no internet, no cloud API), then **syncs-and-purges** attendance to AWS S3 when connectivity returns. It is built to drop into the existing **Datalake 3.0** app. Four open-source ONNX models (9.1 MB total) run on-device in about 51 ms on a host CPU.

Evaluator path (about 60 seconds)

1. Read this page for orientation.
2. Open **`OfflineID_Hackathon7.pptx`**, the 16-slide deck, for the full story.
3. Skim **`01-PROPOSAL.md`** for the solution mapped to the four scoring criteria.
4. Install the **offline APK** from the GitHub Release and try it in airplane mode.

- **Source code:** https://github.com/moneytosms/offlineid
- **Offline release APK (v1.3.0):** https://github.com/moneytosms/offlineid/releases/tag/v1.3.0

What is in this package

File	What it is
`README_FIRST.md` / `.pdf`	This orientation page, start here.
`OfflineID_Hackathon7.pptx`	The presentation, a 16-slide themed deck (mandatory deliverable).
`01-PROPOSAL.md`	Solution overview mapped to Innovation / Feasibility / Scalability / Presentation.
`02-DATALAKE-3.0-INTEGRATION.md`	Exact steps to drop OfflineID into the Datalake 3.0 app, including the iOS engine.
`03-BUILD-OFFLINE-APK.md`	How the standalone offline release APK is built (not debug / Metro).
`PRESENTATION.md`	Slide outline and speaker notes behind the deck.
`README.md`	How to submit on the registration form (zip + link fields) and the full checklist.
`docs/ARCHITECTURE.md`	System design, data flows, and the security model.
`docs/MODEL_PIPELINE.md`	The AI pipeline: detection, alignment, liveness, recognition, preprocessing.
`docs/BENCHMARKS.md`	Model sizes and measured latencies.
`docs/SPEC.md`	Full functional and technical specification.
`docs/SETUP_AND_USAGE.md`	Build, run, and demo walkthrough.

How it satisfies the brief

Constraint	Status
React Native, Android + iOS	RN UI shared; Android native engine ships an offline APK; iOS engine written in Swift (`ios/FaceEngine/`), Xcode wiring pending.
Model footprint ~20 MB	**9.1 MB** total model bundle.
< 1 s recognise + liveness	~51 ms host-CPU pipeline; sub-second on mid-range ARM.
Android 8+ / iOS 12+, 3 GB RAM, no GPU	CPU-only ONNX Runtime (XNNPACK / NNAPI).
> 95% accuracy	MobileFaceNet (LFW 99.5%) + inference-time lighting normalisation.
Offline liveness (blink/smile/turn)	Passive FASNet anti-spoof plus active gesture sequence.
Sync & purge to AWS	Presigned-S3 batch upload, then local purge.
Open-source only	MIT / Apache stack, full source shared.
Low-light operation	Ambient light sensor (TYPE_LIGHT) activates fill-light overlay at < 22 lux (configurable); screen brightness maximised to illuminate face.

What's new in v1.3.0

- **Configurable Preferences** — users can now customize lux threshold, fill-light brightness, haptic feedback, and auto-restart behavior
- **Restructured Settings** — intuitive subviews (Display, Technical, Help) for better organization
- **In-App Help Guide** — gesture explainers, tips for best results, and data privacy info built directly into the app
- **Low-light sensitivity** — lux threshold lowered to 22 (activation) / 35 (deactivation) for better performance in dark environments
- **Screen Wake-Lock** — app prevents device sleep during scanning process
- **Adjustable Zoom** — camera zoom is now configurable and persistent
- **Brand identity** — custom app icon and in-app brand logo (About screen)
- **arm64-v8a only build** — eliminates armeabi-v7a CMake/ninja build issues on Windows; APK ~59 MB

Full instructions and the checklist are in `README.md`.